Learning across the Software Stack: A Research Template for AI and ML

First Author[1] **Second Author**[1] **Third Author**[2]

[1]STAR Lab, George Mason University [2]Collaborating Institution

first@gmu.edu second@gmu.edu third@example.org

ABSTRACT

Learning-based software is evaluated as a model but deployed as a system. This template demonstrates how a STAR Lab paper can connect these two perspectives through a precise problem statement, an explicit method, and an evaluation tied to measurable claims. It includes mathematical notation, a vector overview, pseudocode, a results-table skeleton, citations, and an appendix. The typography and restrained visual identity are designed for long-form AI and machine-learning manuscripts. All prose is instructional sample content; no experiments or empirical findings are reported. Replace the title, authors, sections, and bibliography with the paper's own material.

Keywords machine learning; software engineering; trustworthy systems

Project page: *[project-page-url]*
GitHub: *[github-repository-url]*
Dataset: *[dataset-url]*

1 Introduction

A learned component sits within a chain of data preparation, software construction, compilation, and execution. Explain which part of that chain the paper studies and why the chosen boundary matters. For an AI paper, begin with the learning problem; for an ML systems paper, begin with the workload and deployment constraint.

The research gap. Identify the input, the assumption that fails, and the resulting consequence. A general account of learning, such as Goodfellow et al. (2016), establishes background; a novelty claim requires comparison with specific methods.

The central idea. Describe the mechanism before its implementation details. An illustrative method might combine a task objective with a deployment constraint. Section 2 shows how to express that contract and its limits.

The contribution list should name concrete outputs that the paper actually delivers:

- A problem formulation with explicit assumptions and scope.
- A mechanism whose design choices address the stated limitation.
- Evidence that tests each claim and explains the remaining failures.

2 Problem Formulation and Method

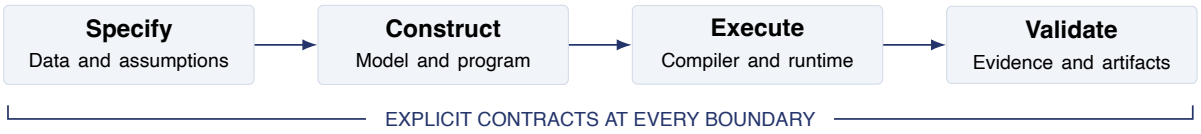


Figure 1. An illustrative research pipeline. Each boundary carries an explicit contract: what enters, what changes, and what is checked. This diagram is a template component, not an evaluated system.

2.1 Define the contract

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$ be a training set and let f_θ denote a model with parameters $\theta \in \mathbb{R}^d$. Write the objective in a form that separates task quality from the system property under study:

$$\theta^\star = \arg\min_{\theta} \; \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}(f_\theta(x_i), y_i) + \lambda \, \Omega(\theta; \mathcal{S}). \tag{1}$$

Here $\mathcal{S}$ specifies the deployment setting, Ω is a chosen penalty, and $\lambda \geq 0$ controls its weight. Define the units and estimation procedure for each term. A penalty is not, by itself, a guarantee that a deployed system satisfies the intended property.

Assumption 2.1 (Fixed evaluation interface)**.** The candidate and reference implementations use the same input schema, preprocessing, and declared numerical precision during evaluation.

2.2 Make the mechanism inspectable

Figure 1 separates specification, construction, execution, and validation. Define each stage's inputs, transformation, and outputs. Explain why a simpler alternative does not address the gap, and distinguish requirements from implementation choices.

Algorithm 1 Illustrative candidate-selection protocol

Require: Candidate set Θ, validation set $\mathcal{V}$, constraint g
Ensure: Best feasible candidate, or explicit failure

```
θ* ← ⊥; s* ← +∞
for each θ ∈ Θ do
    if g(θ, V) ≤ 0 then
        s ← Evaluate(θ, V)
        if s < s* then
            (θ*, s*) ← (θ, s)
        end if
    end if
end for
return θ*
```

Algorithm 1 permits explicit failure when no candidate passes. State the budget, stopping condition, tie-breaking rule, and treatment of invalid outputs. Selection uses validation data; reserve the held-out test set for the final evaluation.

2.3 Bound the claim

Proposition 2.1 (Finite-set selection)**.** *If Θ is finite, all evaluations terminate, and the scores are finite real numbers, Algorithm 1 returns a minimum-score feasible candidate whenever one exists.*

Proof. After each iteration, the stored candidate has the smallest score among the feasible candidates visited so far. The update preserves this invariant. The finite loop visits every candidate, which establishes the result. □

This illustrative statement concerns only the enumerated candidate set. It says nothing about unobserved inputs, global optimality, or deployment security. State those limits alongside any theorem or empirical conclusion.

3 Evaluation Design

Organize the evaluation around questions that test the contribution. For each question, specify the subjects, comparison, metric, and analysis.

3.1 Questions and experimental controls

RQ1: Task quality. Does the method improve the target metric under matched data, tuning budgets, and stopping conditions? **RQ2: System cost.** What latency, memory, and compute costs accompany that change? **RQ3: Mechanism.** Which component explains the observed behavior, and under what conditions does it stop helping?

Report data collection, filtering, splits, and leakage prevention. Record versions, seeds, hardware, and precision settings. Specify whether uncertainty reflects independent runs, resampled inputs, or variation across subjects; these sources answer different questions.

Table 1. A results-table skeleton. Dashes indicate unfilled cells, not measured zeroes. Replace them with results and uncertainty estimates; do not describe improvements until measurements are available.

Configuration	Quality ↑	Latency (ms) ↓	Memory (GB) ↓
Reference implementation	—	—	—
Strong baseline	—	—	—
MethodName	—	—	—
MethodName without constraint	—	—	—

3.2 Results and interpretation

Use Table 1 to report effectiveness and cost. Define the quality metric. For latency, specify batch size, warm-up, synchronization, and the measured percentile. Distinguish peak allocated memory from reserved or resident memory.

Answer each research question and bound its scope. Pair aggregates with failure analysis. Ablations should change one decision at a time; when factors change together, describe the confound.

4 Limitations and Responsible Research

Connect each limitation to an affected claim. Narrow workloads restrict external validity; imperfect labels affect measurement; shared pretraining data complicate independence assumptions. State what was checked and what uncertainty remains.

When security is relevant, specify the attacker, trust boundary, and configuration. Protection principles (Saltzer and Schroeder, 1975) provide context, but a bounded evaluation does not establish general safety.

5 Conclusion

Restate the problem, the central mechanism, and the strongest result that the evidence supports. This document demonstrates the STAR single-column layout; it does not establish an empirical result. Replace this text with a concise conclusion that matches the completed study.

Acknowledgments

Replace this paragraph with verified funding, collaboration, and assistance acknowledgments. This entire section is omitted automatically in anonymous mode.

References

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.

A Reproducibility and Artifact Details

The appendix should carry details that help a reader reproduce or audit the work without interrupting the main argument. Keep assumptions and evidence needed to assess the central claim in the main paper.

Table 2. Suggested artifact manifest. Fill in exact values for the study.

Item	What to record
Source	Commit, license, and commands for each experiment
Environment	OS, accelerator, driver, compiler, and library versions
Data	Provenance, preprocessing, splits, and access conditions
Models	Checkpoints, tokenizer, precision, and inference settings
Compute	Budget, seeds, run count, and failure handling
Evidence	Raw measurements, analysis scripts, and expected outputs

A.1 Example configuration

```
experiment = {
    "model": "replace-with-checkpoint",
    "seed": 0,
    "precision": "float32",
    "split": "held-out-test",
    "record_raw_outputs": True,
}
```

Listing 1. An illustrative configuration schema. Values must be chosen for the actual experiment.

A.2 Editing the manuscript

Edit `paper-info-single.tex` to change the title, authors, affiliation, keywords, and short running title. Edit the files under `sections/single/` to replace the sample content. Keep new notation in `macros.tex`, references in `references.bib`, and illustrations in `figures/`. The shared class controls the visual design.

The bibliography uses BibTeX and author–year citations in this layout. Use `\citet{key}` for a textual citation and `\citep{key}` for a parenthetical citation. Use `\autoref{label}` to create typed references to sections, equations, figures, tables, and algorithms.

A.3 Modes and publication workflow

Add `anonymous` to the document-class options to hide the lab emblem, authors, affiliation, email addresses, acknowledgments, and identifying PDF metadata managed by this class. Inspect the manuscript content, citations, links, and external assets separately for identity disclosures. Add `monochrome` for a grayscale version of the class palette and logo.

Use the lab style for preprints, technical reports, and internal drafts. For a specific venue, use its required class and current author instructions. The section files, bibliography, equations, and vector figures are designed to be reusable, but class-specific helpers need to be replaced when moving to another style. The package's README explains this handoff.